ClassX srl
Via del Tiglio, 91 – Villa Montecchio – 56012 Calcinaia (PI) – ITALY
tel: 0587/488128 Email: support@classx.it Web: www.classxcg.com

LBControl™

the LiveBoard scripting and control API

Quick guide to the various ways to control LiveBoard and its contents.

Rev 1.5 – 18.08.2021

© ClassX 2008/2021

License conditions

Copyright

The following API specifications have been designed, developed and maintained by ClassX srl All rights are reserved worldwide. LBControl™ is a registered trademark of ClassX.

Obligations

The usage of the following API to develop and integrate third party application is free from any obligations. ClassX does not assume and cannot assume any kind of responsibility deriving from proper or improper usage of this protocol.

Support

This API is distributed as-is, without any warranty, expressed or implied. ClassX reserves the exclusive right to change the protocol and its specifications, the objective and the usage at any time without any notice.
ClassX is not obligated in any way to give any kind of service support to the user of the following protocol.

Disclaimer of warranties

ClassX MAKES NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, AS TO ANY OTHER MATTERS, INCLUDING BUT NOT LIMITED TO NON-INFRINGEMENT OF THIRD PARTY RIGHTS, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. ClassX DOES NOT AND CANNOT WARRANT THE PERFORMANCE OR RESULTS YOU MAY OBTAIN BY USING THE SOFTWARE.

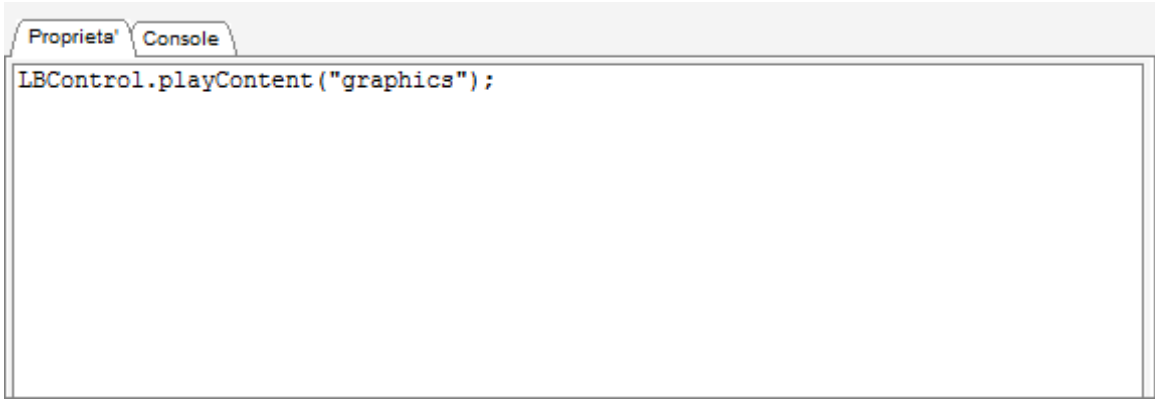
IN NO EVENT WILL ClassX BE LIABLE TO YOU FOR ANY CONSEQUENTIAL, INDIRECT, INCIDENTAL, PUNITIVE OR SPECIAL DAMAGES, INCLUDING ANY LOST PROFITS OR LOST SAVINGS, EVEN IF A REPRESENTATIVE OF ClassX OR ANY SUPPLIER HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES, OR FOR ANY CLAIM BY ANY THIRD PARTY.

1 What is LBControl

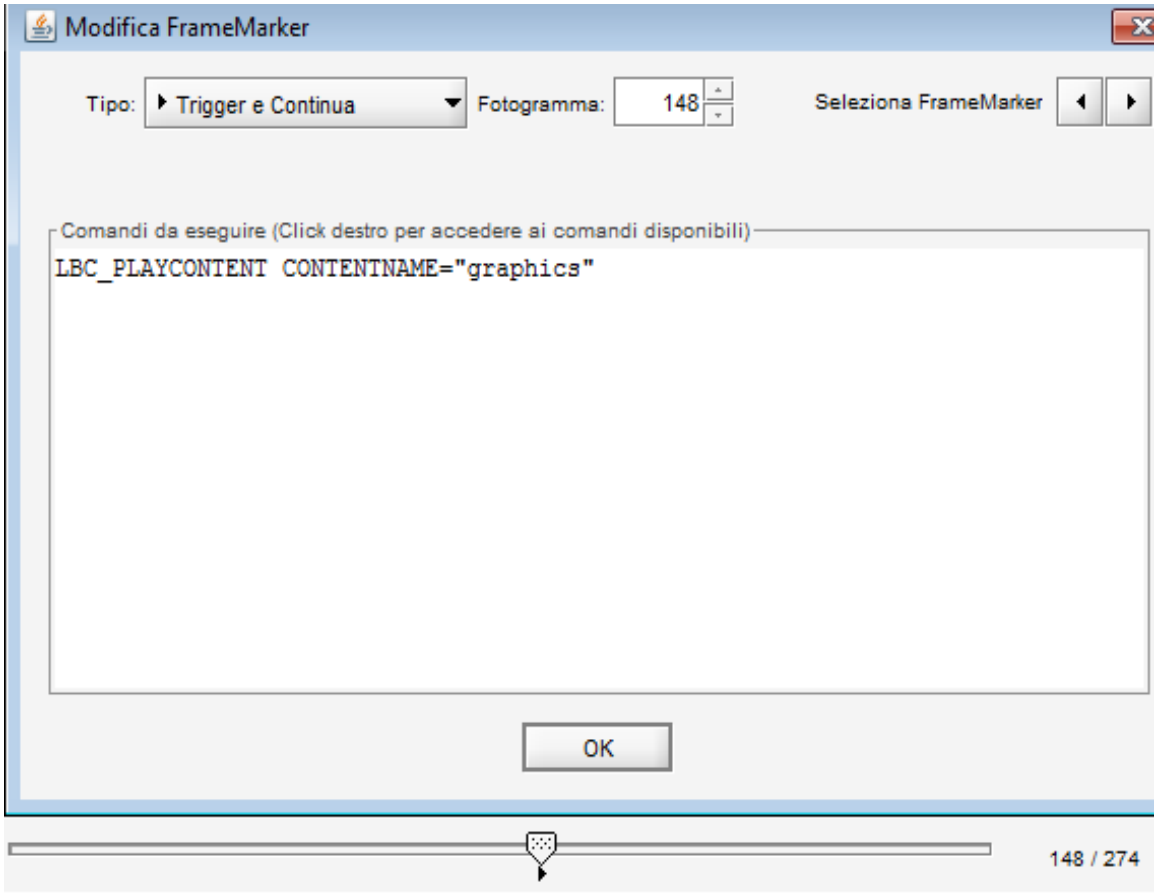
LiveBoard and all its variations (Light Edition, Stand Alone, SCG, etc..) offers to developers and high-end users the possibility to be controlled in many different ways.

LBControl is a lightweight interface designed to control the LiveBoard contents through three different ways:

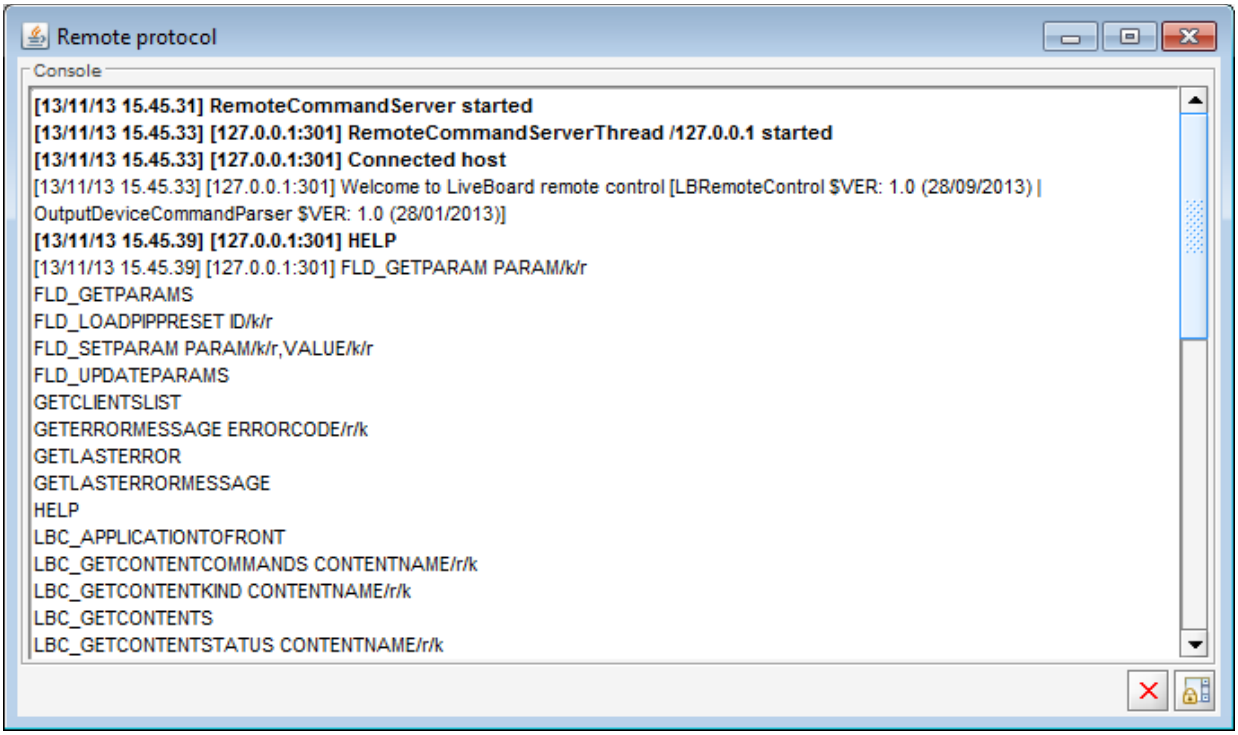
- **Script** object (not supported by LiveBoard LE), Java-like, BeanShell-based scripting language;



- Command line interpreter called **TemplateActions™**;



- Asynchronous **TCP/IP** interpreter;



Basically, users of LiveBoard can count on the LBControl interface that impersonates the operator and all he can do operatively with LiveBoard.

LiveBoard SCG has an additional StealthCG protocol that has even more features than the LBControl and it is especially addressed to high-end developers and OEM integrators. The description of the StealthCG protocol is beyond the scope of this document and we have reserved it ad-hoc documentation in the ClassX Devkit.

In the following scheme we can see a reference of the controlling features of the LiveBoard variations:

	Script	Template Actions	TCP/IP Protocol	StealthCG
LiveBoard	X	X	X	-
LiveBoard LE	-	X	X	-
LiveBoard SCG	X	X	X	X



The LiveBoard Script engine is based on top of the BeanShell engine. More on **BeanShell** can be found at the page http://www.beanshell.org/
BeanShell dynamically executes standard Java syntax and extends it with common scripting conveniences such as loose types, commands, and method closures like those in Perl and JavaScript. In short, BeanShell is dynamically interpreted Java, plus a scripting language and flexible environment all rolled into one clean package.
The LiveBoard scripting engine extends the functionalities of **BeanShell** by giving direct access to some specialized interfaces, designed for the control of many aspects of LiveBoard using the variable "**LBControl**".

2 How to use the LBControl interface

The user might understand the differences in invoking LBControl commands from Script, TemplateActions and TCP/IP environment. He will notice that there is practically 1:1 match between the LBControl API and the interpetered counterpart. Most of the LBControl commands have been replicated to the TemplateActions™ supported commands so that developers can easily access LBControl functionalities at no cost. Their syntax is more "script friendly" and quite similar to the original LBControl commands and so the same applies to the expected behaviour.

Let's see a quick example: **playing a LiveBoard content called "graphics"**:

- **Script**

```
it.classx.remoteprotocol.RemoteCommandResult r = LBControl.playContent("graphics");
print(r);
```

  (r) will return the result of the operation.

- **TemplateActions™**

```
LBC_PLAYCONTENT "graphics"
```

- **TCP/IP**

```
(send) > LBC_PLAYCONTENT "graphics"
(recv) < OK
```

More complex example: **getting a value from a template called "graphics"**:

- **Script**

```
it.classx.remoteprotocol.RemoteCommandResult r =
LBControl.getTemplateObjectValue("graphics","subtitle");

print(r.getResult());
```

  r.getResult() will return the value of the object "subtitle" as String.

- **TemplateActions™**

```
LBC_GETTEMPLATEOBJECTVALUE "graphics","subtitle"
```

  The TemplateActions command will be correctly executed, but you'll have no chance to get the result.

- **TCP/IP**

```
(send) > LBC_GETTEMPLATEOBJECTVALUE "graphics","subtitle"
(recv) < "value_of_the_field"
```

For a complete API reference of the supported methods and interfaces of the LBControl API, please refer to the accompanying html JavaDoc here.

3 Exiting safely fom a running LBScript

When executing an LBScript, **LiveBoard** allocates a separate **thread** for every script execution that may happen both by clicking on the content button or through an external event. At the same time the script can be **stopped** in the same way. LiveBoard can also **automatically** stop scripts after a certain amount of time if they do not exit automatically. This feature is enabled with the "timeout" setting of the script.

In any case, script implementors are required to check the status of the special *boolean* variable called "**LBScript.interrupted**" in order to exit safely from loops and long-time execution loops. The LiveBoard scripting engine will set/reset this variable automatically on content play/stop/timeout.

The pseudo code for such implementation is the following:

```
while(!LBScript.interrupted)
{
        // do your job here
}

or

for(int i=0; i<100; i++)
{
        if (LBScript.interrupted) break;
        // do your job here
}
```

4 Built-In commands

The BeanShell default commands have been extended with the utility methods listed below:

- **`TCPIPSendCommand(String hostName, int hostport, String command, boolean waitReply)`**

  Send a command through the TCP/IP connection established on the given host name and port. If <waitreply> is true this will wait the command answer and return it.

- **`sleep(long millis)`**

  Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds, subject to the precision and accuracy of system timers and schedulers.

- **`print(Object arg)`**

  Print the string value of the argument, which may be of any type. If Beanshell is running interactively, the output will always go to the command line, otherwise it will go to System.out.

  Most often the printed value of an object will simply be the Java toString() of the object. However if the argument is an array the contents of the array will be (recursively) listed in a verbose way.

  Note that you are always free to use System.out.println() instead of print().

- **`include(String fileName)`**

  This command emulates the #include<> preprocessor command of c,c++ it also avoid to re-include files already included. A timestamp system allows to re-include only changed files.

- **`drawSharedFrame(String sharedFrameName, int shareFrameW, int sharedFrameH, Image image)`**

  Draw the given image on the ShareFrame identified by the sharedFrameName, shareFrameW and shareFrameH. If the image size is greater then the SharedFrame, it is automatically adapted to fit the area keeping its aspect.

5 LBMicroServer

LBMicroServer is distributed with the ClassX Broadcast applications and it is runnable by Script of LiveBoard. It is a simple application used to execute TCP-IP commands received by external tools. LBMicroServer can be used to play and to stop the LiveBoard contents from remote applications.

NOTE: LBMicroServer has been **deprecated** and replaced by LBControl.

To use LBMicroServer from LiveBoard it is necessary to instance the class by a Script content of LiveBoard. The source code should be similar to the following:

```
import script.lbmicroserver.LBMicroServer;

if (frame == void)
{
	int port = 102;

	// instance of LBMicroServer
	LBMicroServer microServer = new LBMicroServer(LBControl, port);

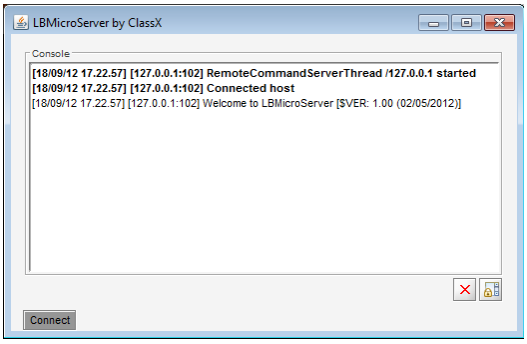
	// create host frame
	frame = new javax.swing.JFrame();
	frame.setTitle("LBMicroServer by ClassX");
	frame.setSize(microServer.getSize());
	java.awt.Insets insets = frame.getInsets();
	frame.setSize(frame.getWidth() + insets.left + insets.right, frame.getHeight() + insets.top +
insets.bottom);

	// add LBMicroServer to frame
	frame.getContentPane().add(microServer);

	// show
	frame.setVisible(true);
}
else
{
	frame.setVisible(!frame.isVisible());
}
```

As we can see from the source code, the constructor of LBMicroServer class has 2 arguments: LBControl and the tcp-ip port number.

When the Script content is executed, LBMicroServer shows a simple user interface with a console used to show the received commands and the relative returned messages. To receive the commands from external tools it is necessary press the "Connect" button.



Automatically LBMicroServer creates a log file into the application folder, named FTSERVER_XXX-XX-XX.log (XXXX-XX-XX = date of the day) where are stored all the received commands with the relative returned messages.
The following are the supported command list:

- **PLAY "ContentName"**

Play the LiveBoard content named “ContentName”. If the content doesn't exist, LBMicroServer returns an error “Error 3001” (Content not found) , else “Ok”.

- **`STOP "ContentName"`**

  Stop the LiveBoard content named “ContentName”. If the content doesn't exist, LBMicroServer returns an error “Error 3001” (Content not found) , else “Ok”.

- **`GETCONTENTSTATUS "ContentName"`**

  Return the status of te content named “ContentName”. If the content doesn't exist, LBMicroServer returns an error “Error 3001” (Content not found) , else it returns:

  - “0” if the content is stopped;
  - “1” if the content is playing;
  - “2” if the content is paused.

6 PlugIns Command list

ClassX srl
Via del Tiglio, 91 – Villa Montecchio – 56012 Calcinaia (PI) – ITALY
tel: 0587/488128 Email: support@classx.it Web: www.classxcg.com

PlugIns™ commands list

Protocol Specifications
Rev 1.4 - 18-05-2021

© ClassX 2021

6.1 Common commands

Each plugIn supports a set of remote commands that allows to change its appearance, position and settings. The plugIns have some common properties as position, dimension described in this section, while others properties depend from their implementation and will be illustrated into the related chapters.

SETPOSITION <X>,<Y>

Move the plugIn to the screen location X,Y. The parameter with value equals to -1 will be ignored.

Example: move the content to X = 100 coordinate keeping the current Y coordinate.

```
>SETPOSITION 100,-1;
<OK
```

SETSIZE <WIDTH>,<HEIGHT>

Resize the plugIn to the given WIDTH and HEIGHT. The parameter with value equals to -1 will be ignored.

Example: set the content width to 100 keeping the current height.

```
>SETSIZE 100,-1;
<OK
```

{DEPRECATED} SETVISUALPRIORITY <VALUE>

Set the plugIn visual priority in a range value [0,10000]: 0 is the lower layer and 10000 is the higher layer.

NOTE: This command does not produce any result.

Opening "**Remote Control Tester**" in ClassX distribution and **connecting** to LiveBoard, it is possible to view the list of available commands for each Plugin.

Just insert this command in the "**Command**" area:

LBC_GETCONTENTCOMMANDS <value>

6.2 Logo

SETFILENAME <FILE>

Set logo file name.

Example
```
>SetFileName \"C:\\\\Pictures\\\\image.png\"
<OK
```

SETLOGOPROPERTY <PLAYSPEED>, <PLAYMODE>, <FIELDFLAGS>, <USEALPHA>, <FPS>

Set the Logo properties. You can set one or more properties in the same command:

PLAYSPEED speed value percentage over original speed, used to play the logo [0..200]. The default is 100 (the original speed);
PLAYMODE mode used to play the logo [0..9]. The default value is 3 [PLAY_LOOP].

- 0 = still first frame [PLAY_STILLFIRST];
- 1 = still last frame [PLAY_STILLLAST];
- 2 = play one time only [PLAY_ONCE];
- 3 = loop animation [PLAY_LOOP];
- 4 = reverse loop animation [PLAY_REVLOOP];
- 5 = ping-pong animation [PLAY_PINGPONG];
- 6 = play one time only automatically [PLAY_ONCE_AUTO];
- 7 = loop animation automatically [PLAY_LOOP_AUTO];
- 8 = reverse loop animation automatically [PLAY_REVLOOP_AUTO];
- 9 = ping-pong animation automatically [PLAY_PINGPONG_AUTO];

FIELDFLAGS field handling [0 = keep original interlace fields, 1 = de-interlace frame];
USEALPHA use the alpha information of the logo if present (default is TRUE);
FPS play speed of the animation expressed in frames per second.

Example:
```
>SetLogoPropery PLAYSPEED=50
<OK
```
Set the play speed of the Logo to the 50% of the normal speed.

```
> SetLogoPropery PLAYMODE=5,USEALPHA
<OK
```
Ping-pong animation and maintain the frames alpha value.

6.3 Timer

SETSTARTTIME <VALUE>

Set the Timer start time in seconds or milliseconds. To set it in seconds it is sufficient to give it a numeric value, or the same value followed by “s”, while to set it in milliseconds the numeric value must be followed by “m”.

Example 1: set the start time to 20 seconds.

```
>SetStartTime \"20\"
<OK
```

Example 2: set the start time to 20 seconds.

```
>SetStartTime \"20s\"
<OK
```

Example 3: set the start time to 20000 milliseconds.

```
>SetStartTime \"20m\"
<OKSETENDTIME <VALUE>
```

Set the Timer end time in seconds.

Example: set the end time to 60 seconds.

```
>SetEndTime 60
<OK
```

SETTIMERDIRECTION <[UP|DOWN]>

Set the timer direction: set UP for a timer and DOWN for a countdown
Example: setup the timer for a countdown

```
>SetTimerDirection DOWN
<OK
```

RESETTIMER

Reset the timer to start time seconds.

STARTTIMER

Start the timer count.

STOPTIMER

Stop the timer count

GETTIMEMILLIS

Return the current timer time in milliseconds

SETTIMERFORMAT

Set the time format of the timer. The only supported formats are the following:

- `"s:tt"` - seconds and milliseconds.
- `"ss:tt"` - seconds and milliseconds with 2 cyphers for the seconds always visible.
- `"s"` - seconds.
- `"ss"` - seconds with 2 cyphers always visible.
- `"sss"` -seconds with 3 cyphers always visible.
- `"m:ss"` - minutes and seconds.
- `"mm:ss"` - minutes and seconds with 2 cyphers for the minutes always visible.
- `"mmm:ss"` - minutes and seconds with 3 cyphers for the minutes always visible.
- `"HH:mm:ss"` - hours, minutes and seconds.
- `"HH:mm"` - hours and minutes.
- `"HH:mm:ss:tt"` - hours, minutes, seconds and tenths of second.

Example: setup the timer to display hours, minutes, seconds and tenths of second.

```
>SetTimerFormat \"HH:mm:ss:tt\"
<OK
```

6.4 Horizontal and Styled Ticker

ADDTICKERTEXT <TEXT>, <LOOPS>

Set the text to scroll into the Ticker defining also the loops numbers. Use –1 for infinite loop.

Example: set the text and loops to 3 times

```
>AddTickerText \"Tonight at 21.30: Music\", 3
<OK
```

CLEARPLAYLIST [<IMMEDIATELY>]

Clear the Ticker files playlist. If IMMEDIATELY is true the Ticker execution will be also interrupted.

Example:

```
>ClearPlaylist
<OK

>ClearPlaylist IMMEDIATELY=true
<OK
```

ADDPLAYLISTFILE <FILEPATH,MAXRECORDS_NUM,MAXRECORDS_FROMLAST>

Add the given file to the playlist at the last position. The file path may refer to either local or remote URL file. If MAXRECORDS_NUM is greater than 0, limit the number of records imported from the file. In this case MAXRECORDS_FROMLAST defines where the limit is applied: from the first or the last records.

Example:

```
>AddPlayListFile \"C://Datasource//tg.txt\"
<OK

>AddPlayListFile \"http://www.classx.it//prova.txt\"
<OK
```

ADDPLAYLISTFILES <FILEPATHS=<FILEPATH, MAXRECORDS_NUM, MAXRECORDS_FROMLAST>>

Add the given files to the playlist at the last position. The file path may refer to either local or remote URL file. If MAXRECORDS_NUM is greater than 0, limit the number of records imported from each file. In this case MAXRECORDS_FROMLAST defines where the limit is applied: from the first or the last records.

Example:

```
>AddPlaylistFiles
FILEPATHS=\"FILEPATH=\\\"C:\\\\\\\\\ClassX_Applications_v6\\\\\\\\shared\\\\\\\\datasource\\\\\\
\\class_cnbc\\\\\\\\tickers\\\\\\\\ita\\\\\\\\DAX30_INDEX\\\\\\\\2268719894846_Adidas
Ag.rss\\\"\\r\""
<RCVD: Ok
```

RESETPLAYLIST

Reset the playlist execution forcing the Ticker to restart by the first playlist file.

Example:

```
>ResetPlaylist
<OK
```

GETCURRENTTEXTDURATION

Return the scrolling time in milliseconds of the current text. The duration depends by the device resolution, the ticker width and the font size. Call this command while the text is scrolling.

Example:

```
>GetCurrentTextDuration
<21000
```

SETTICKERPROPERTY <SPEED> , <TEXTUPPERCASE>, <TEXTSEPARATOR>, <AUTOREMOVE>

Set the ticker properties. You can set one or more properties in the same command:

SPEED the ticker speed (px/s) [1, 30];
TEXTUPPERCASE true if convert to uppercase the ticker text;
TEXTSEPARATOR the ticker text separator;
AUTOREMOVE stop the ticker on finish the playlist execution;

Examples:

```
>SETSTYLEDTICKERPROPERTY "SPEED=8"
<OK

>SETSTYLEDTICKERPROPERTY "TEXTUPPERCASE=TRUE"
<OK

>SETSTYLEDTICKERPROPERTY "TEXTSEPARATOR=\" ~ \""
<OK

>SETSTYLEDTICKERPROPERTY "AUTOREMOVE=\"TRUE\""
<OK
```

6.5 Styled Ticker Horizontal

SETSTYLEDTICKERPROPERTY <SPEED> , <TEXTUPPERCASE>, <TEXTSEPARATOR>, <AUTOREMOVE>, <TEXTDIRECTIONMODE>

Set the ticker properties. You can set one or more properties in the same command:

SPEED the ticker speed (px/s) [1, 30];
TEXTUPPERCASE true if convert to uppercase the ticker text;
TEXTSEPARATOR the ticker text separator;
AUTOREMOVE stop the ticker on finish the playlist execution;
TEXTDIRECTIONMODE the ticker text direction [AUTOMATIC | LEFT_TO_RIGHT | RIGHT_TO_LEFT];
Note. AUTOMATIC: Automatically detect the text direction LTR or RTL

Examples:

```
>SETSTYLEDTICKERPROPERTY "TEXTDIRECTIONMODE=AUTOMATIC"
<OK

>SETSTYLEDTICKERPROPERTY "TEXTDIRECTIONMODE=LEFT_TO_RIGHT"
<OK

>SETSTYLEDTICKERPROPERTY "TEXTDIRECTIONMODE=RIGHT_TO_LEFT"
<OK
```

6.6 MediaText

SETTEXT <TEXT>

Set the text to display into the MediaText area.

Example:

```
>SetText \"Tonight 21.30: Blue Diamond\"
<OK
```

GETTEXT

Get the text displayed into the MediaText area.

Example:

```
>GetText
<"TYPE=\"Tonight 21.30: Blue Diamond\""
```

SETBACKGROUNDCOLOR <A>,<R>,<G>,<B>

Set the MediaText background color supporting also the alpha channel

Example:

```
>SetBackgroundColor A=255,R=255,G=0,B=0
<OK
```

SETFOREGROUNDCOLOR <A>,<R>,<G>,<B>

Set the MediaText foreground color supporting also the alpha channel.

Example:

```
>SetForegroundColor A=255,R=255,G=0,B=0
<OK
```

SETFOREGROUNDCOLOR <A>,<R>,<G>,<B>

Set the MediaText foreground color supporting also the alpha channel.

Example:

```
>SetForegroundColor A=255,R=255,G=0,B=0
<OK
```

6.7 TextSlideShow

SETFILENAME <FILE>

Set the TextSlideShow file to process and display.

Example:

```
>SetFileName \"C://Dataource/plugIn.csv\"
<OK
```

6.8 SimpleMedia

SETFILENAME <FILE>,<LOOPS>,<HQ>,<FILLALPHA>,<USEALPHA>

Set the SimpleMedia filename to play and enable loop and other execution properties. The loop parameter is not required and its default value is false. Use HQ to resize the media with best quality. The default SimpleMedia screen location is (0,0) and its size is the same of the loaded file.

Example:

```
>SetFileName \"C:\\\\Video\\video_loop.avi\",LOOP,HQ
<OK

>SetFileName \"C:\\\\Video\\video_loop.avi\",HQ
<OK
```

SETMEDIAPROPERTY <LOOP>, <HQ>, <FILLALPHA>, <USEALPHA>, <SCALETOFIT>, <FIELDPOLARITY>, <SYNCHTOVIDEOREFRESH>, <AUDIOONLY>, <AUDIODELAY>, <VOLUME>, <PAUSEFIRST>, <PAUSELAST>, <MARKINMILLIS>, <MARKOUTMILLIS>, <MARKIN>, <MARKOUT>, <MEDIATIMEFRAME>, <MEDIATIMEMILLIS>, <PIXELASPECTRATIO>

Set the SimpleMedia properties. You can set one or more properties in the same command:

LOOP play the media in loop mode [TRUE | FALSE];
HQ use the best quality when scaling the media [TRUE | FALSE];
FILLALPHA fill the alpha channel of the media [TRUE | FALSE];
USEALPHA compose the alpha channel of the media with the background [TRUE | FALSE];
SCALETOFIT scale the media to fit the size assigned to the plugin [TRUE | FALSE];
FIELDPOLARITY **(DEPRECATED)** set the media video fields polarity [0 = EVEN, 1 = ODD, 2 = NONE];
FIELDFLAGS set the media video fieldflags mode [0 = NONE, 1 = DEINTERLACE, 2 = EVEN_DOMINANT, 3 = ODD_DOMINANT];
SYNCHTOVIDEOREFRESH synchronize the media with the output device video refresh freq [TRUE | FALSE];
AUDIOONLY use only the audio of the media (no video) [TRUE | FALSE];
AUDIODELAY set A/V delay;
VOLUME [0..255] set the volume of the audio.
PAUSEFIRST enable or disable the pause at first frame [TRUE | FALSE];
PAUSELAST enable or disable the pause at last frame [TRUE | FALSE];
MARKINMILLIS set the mark in [The media time in milliseconds];
MARKOUTMILLIS set the mark out [The media time in milliseconds];
MARKIN set the mark in [The media time in frames];
MARKOUT set the mark out [The media time in frames];
MEDIATIMEFRAME set the media time [The media time in frames];
MEDIATIMEMILLIS set the media time [The media time in milliseconds];
PIXELASPECTRATIO set the pixel aspect ratio from preset or ratio in floating value

[
FREE,
SQUARE,
PAL_NTSC_4_3,
PAL_NTSC_16_9,
PAL_4_3_DV,
PAL_16_9_DV,
NTSC_4_3_DV,
NTSC_16_9_DV
]

Example:

```
>SetMediaProperty LOOP
<OK
```

enable the play loop

```
>SetMediaProperty LOOP=FALSE,VOLUME=180
<OK
```

disable play loop and set clip volume to 180

```
>SetMediaProperty LOOP, HQ, FILLALPHA, USEALPHA, SCALETOFIT, FIELDPOLARITY=1,
SYNCHTOVIDEOREFRESH, AUDIOONLY, AUDIODELAY=600, VOLUME=180
<OK

>SetMediaProperty  MARKIN=120
<OK

>SetMediaProperty  MARKOUT=200
<OK

>SetMediaProperty  MARKINMILLIS=4200
<OK

>SetMediaProperty  MARKOUTMILLIS=6200
<OK

>SetMediaProperty  FIELDFLAGS=0
<OK
```

GETMEDIAPROPERTY <LOOP>, <HQ>, <FILLALPHA>, <USEALPHA>, <SCALETOFIT>, <FIELDPOLARITY>, <SYNCHTOVIDEOREFRESH>, <AUDIOONLY>, <AUDIODELAY>, <VOLUME>, <PAUSEFIRST>, <PAUSELAST>, <MARKINMILLIS>, <MARKOUTMILLIS>, <MARKIN>, <MARKOUT>, <MEDIATIMEFRAME>, <MEDIATIMEMILLIS>, <PIXELASPECTRATIO>, <DURATION>, <TOTFRAMES>, <ALL>

Get the SimpleMedia properties. You can get one or more properties in the same command:

LOOP get media loop mode [TRUE | FALSE];
HQ get media best quality flag [TRUE | FALSE];
FILLALPHA get the fill alha channel flag [TRUE | FALSE];
USEALPHA get the compose the alpha channel flag of the media [TRUE | FALSE];
SCALETOFIT get the scale to fit flag of the media size assigned to the plugin [TRUE | FALSE];
FIELDFLAGS get the media video fieldflags mode [0 = NONE, 1 = DEINTERLACE, 2 = EVEN_DOMINANT, 3 = ODD_DOMINANT];
SYNCHTOVIDEOREFRESH get the synchronize the media with the output device video refresh freq flag [TRUE | FALSE];
AUDIOONLY get the use only the audio of the media flag [TRUE | FALSE];
AUDIODELAY get A/V delay;
VOLUME [0..255] get the volume of the audio.
PAUSEFIRST get the the pause at first frame status [TRUE | FALSE];
PAUSELAST get the pause at last frame status [TRUE | FALSE];
MARKINMILLIS get the mark in [The media time in milliseconds];
MARKOUTMILLIS get the mark out [The media time in milliseconds];
MARKIN get the mark in [The media time in frames, -1 if no set];
MARKOUT get the mark out [The media time in frames, -1 if no set];
MEDIATIMEFRAME get the current media time [The media time in frames];
MEDIATIMEMILLIS get the current media time [The media time in milliseconds];
PIXELASPECTRATIO get the pixel aspect ratio floating value
DURATION get the media duration [The media duration in milliseconds];
TOTFRAMES get the media duration in frames [The media duration in frames];
ALL retrieve all properties;

Examples:

```
>GETMEDIAPROPERTY FIELDFLAGS
<FIELDFLAGS=0

>GETMEDIAPROPERTY FIELDFLAGS, LOOP, HQ, FILLALPHA, USEALPHA, SCALETOFIT, SYNCHTOVIDEOREFRESH,
AUDIOONLY, AUDIODELAY, VOLUME, MARKIN, MARKOUT, MARKINMILLIS, MARKOUTMILLIS, MEDIATIMEFRAME,
MEDIATIMEMILLIS, DURATION, TOTFRAMES
<FIELDFLAGS=0,LOOP="FALSE",HQ="TRUE",FILLALPHA="FALSE",USEALPHA="FALSE",SCALETOFIT="FALSE",SYNC
HTOVIDEOREFRESH="TRUE",AUDIOONLY="FALSE",AUDIODELAY=200,VOLUME=255,MARKIN=0,MARKOUT=0,MARKINMIL
LIS=-1,MARKOUTMILLIS=-1,             MEDIATIMEFRAME=0,MEDIATIMEMILLIS=0,PIXELASPECTRATIO=8.333,
DURATION=1000,TOTFRAMES=25

>GETMEDIAPROPERTY ALL
<FIELDFLAGS=0,LOOP="FALSE",HQ="TRUE",FILLALPHA="FALSE",USEALPHA="FALSE",SCALETOFIT="FALSE",SYNC
HTOVIDEOREFRESH="TRUE",AUDIOONLY="FALSE",AUDIODELAY=200,VOLUME=255,MARKIN=0,MARKOUT=0,MARKINMIL
LIS=-1,MARKOUTMILLIS=-
1,MEDIATIMEFRAME=0,MEDIATIMEMILLIS=0,PIXELASPECTRATIO=8.333,DURATION=1000,TOTFRAMES=25
```

ADDFRAMEMARKER <FRAME>, <TYPE>, <LOOPS>

Add a FrameMarker at a specific frame.

FRAME frame number for adding the FrameMarker
TYPE FrameMarker type, it can assume one of the following values: TRIGGER_AND_PAUSE, TRIGGER_AND_CONTINUE, TRIGGER_AND_LOOP_START, TRIGGER_AND_LOOP_END, TRIGGER_AND_MARKIN, TRIGGER_AND_MARKOUT
LOOPS number of loops. It is valid only if the FrameMarker type is: TRIGGER_AND_LOOP_END

Examples:

```
>LBC_SENDCONTENTCOMMAND "SIGLA", "ADDFRAMEMARKER 185, TRIGGER_AND_CONTINUE"
<Ok

>LBC_SENDCONTENTCOMMAND "SIGLA", "ADDFRAMEMARKER 268, TRIGGER_AND_LOOP_END, 3"
<Ok
```

REMOVEFRAMEMARKER <FRAME>

Remove the FrameMarker at a specific frame.

FRAME frame number for deleting the FrameMarker

Examples:

```
>LBC_SENDCONTENTCOMMAND "SIGLA", "REMOVEFRAMEMARKER 185"
<Ok
```

GETFRAMEMARKERS

Return the list of the FrameMarker .

Examples:

```
>LBC_SENDCONTENTCOMMAND "SIGLA", "GETFRAMEMARKERS"
<FRAME=38,TYPE="TRIGGER_AND_MARKIN"
 FRAME=130,TYPE="TRIGGER_AND_LOOP_START"
 FRAME=240,TYPE="TRIGGER_AND_CONTINUE"
 FRAME=417,TYPE="TRIGGER_AND_PAUSE"
 FRAME=1583,TYPE="TRIGGER_AND_LOOP_END", LOOPS=3
 FRAME=1696,TYPE="TRIGGER_AND_MARKOUT"
```

SETCROPINFO <ENABLE>, <PARAM0>, <PARAM1>, <PARAM2>, <PARAM3>

Set the crop info. You can set one or more properties in the same command:

ENABLE set the crop enable [TRUE | FALSE];
PARAM0
PARAM1
PARAM2
PARAM3 are the crop integer values in pixel;

This command respects the HTML#margins standard.

If you set 4 values are applied with order top/right/bottom/left
If you set 3 values are applied with order top/left/bottom
If you set 2 values the first value is applied to top/bottom and the second is applied to left/right
If you set 1 values are applied in all sides

Examples:

```
// ENABLE="TRUE",TOP=10,RIGHT=11,BOTTOM=12,LEFT=13
>SETCROPINFO ENABLE 10 11 12 13
<OK

// ENABLE="TRUE",TOP=10,RIGHT=0,BOTTOM=12,LEFT=13
```

```
>SETCROPINFO ENABLE 10 11 12
<OK

// ENABLE="TRUE",TOP=15,RIGHT=20,BOTTOM=15,LEFT=20
>SETCROPINFO ENABLE 15 20
<OK

// ENABLE="TRUE",TOP=10,RIGHT=10,BOTTOM=10,LEFT=10
>SETCROPINFO ENABLE 10
<OK
```

GETCROPINFO <ENABLE>, <TOP>, <RIGHT>, <BOTTOM>, <LEFT>, <ALL>

Set the crop info. You can set one or more properties in the same command:

ENABLE set the crop enable [TRUE | FALSE];
TOP get the crop top integer value in pixel;
RIGHT get the crop right integer value in pixel;
BOTTOM get the crop bottom integer value in pixel;
LEFT get the crop left integer value in pixel;
ALL retrieve all properties;

Examples:

```
>LBC_SENDCONTENTCOMMAND "SIGLA", "GETCROPINFO ENABLE TOP RIGHT BOTTOM LEFT"
<ENABLE="TRUE",TOP=10,RIGHT=0,BOTTOM=12,LEFT=11

>LBC_SENDCONTENTCOMMAND "SIGLA", "GETCROPINFO ALL"
<ENABLE="TRUE",TOP=10,RIGHT=0,BOTTOM=12,LEFT=11
```

6.9 Image

SETFILENAME <FILE>,<USEALPHA>,<HQ>

Set the Background filename to display. The USEALPHA parameter is not required and its default value is false. Use HQ to resize the media with best quality.
The default Background screen location is (0,0) and its size is the same of the loaded file.

Example:

```
>SetFileName \"C:\\\\Video\\video_loop.avi",ALPHA,HQ
<OK

>SetFileName \"C://Video/video_loop.avi",HQ
<OK
```

6.10 MediaPlayer

CLEARPLAYLIST

Clear the MediaPlayer files playlist.

Example:

```
>ClearPlaylist
<OK
```

ADDPLAYLISTFILE <FILEPATH>

Add the given file to the playlist at the last position. The file path may refer to either local or remote URL file.

Example:

```
>AddPlayListFile \"C://Datasource//tg.txt\"
<OK

>AddPlayListFile \"http://www.classx.it//prova.txt\"
<OK
```

RESETPLAYLIST

Reset the playlist execution forcing the MediaPlayer to restart by the first playlist file.

Example:

```
>ResetPlaylist
<OK
```

SETMEDIAPROPERTY <LOOP>, <HQ>, <FILLALPHA>, <USEALPHA>, <SCALETOFIT>, <FIELDPOLARITY>, <SYNCHTOVIDEOREFRESH>, <AUDIODELAY>, <VOLUME>

Set the SimpleMedia properties. You can set one or more properties in the same command:

LOOP play the media in loop mode;
HQ use the best quality when scaling the media;
FILLALPHA fill the alpha channel of the media;
USEALPHA compose the alpha channel of the media with the background;
SCALETOFIT scale the media to fit the size assigned to the plugin;
FIELDPOLARITY **(DEPRECATED)** set the media video fields polarity [0 = EVEN, 1 = ODD, 2 = NONE];
FIELDFLAGS set the media video fieldflags mode [0 = NONE, 1 = DEINTERLACE, 2 = EVEN_DOMINANT, 3 = ODD_DOMINANT];
SYNCHTOVIDEOREFRESH synchronize the media with the output device video refresh freq;
AUDIODELAY set A/V delay;
VOLUME [0..255] set the volume of the audio.

Example:

```
>SetMediaProperty LOOP
<OK
```

enable the play loop

```
>SetMediaProperty LOOP=FALSE,VOLUME=180
<OK
```

disable play loop and set clip volume to 180

```
>SetMediaProperty LOOP, HQ, FILLALPHA, USEALPHA, SCALETOFIT, FIELDPOLARITY=1,
SYNCHTOVIDEOREFRESH, AUDIODELAY=600, VOLUME=180
<OK
```

SETCROPINFO <ENABLE>, <PARAM0>, <PARAM1>, <PARAM2>, <PARAM3>

Set the crop info. You can set one or more properties in the same command:

ENABLE set the crop enable [TRUE | FALSE];
PARAM0
PARAM1
PARAM2
PARAM3 are the crop integer values in pixel;

This command respects the HTML#margins standard.

If you set 4 values are applied with order top/right/bottom/left
If you set 3 values are applied with order top/left/bottom
If you set 2 values the first value is applied to top/bottom and the second is applied to left/right
If you set 1 values are applied in all sides

Examples:

```
// ENABLE="TRUE",TOP=10,RIGHT=11,BOTTOM=12,LEFT=13
>SETCROPINFO ENABLE 10 11 12 13
<OK

// ENABLE="TRUE",TOP=10,RIGHT=0,BOTTOM=12,LEFT=13
>SETCROPINFO ENABLE 10 11 12
<OK

// ENABLE="TRUE",TOP=15,RIGHT=20,BOTTOM=15,LEFT=20
>SETCROPINFO ENABLE 15 20
<OK

// ENABLE="TRUE",TOP=10,RIGHT=10,BOTTOM=10,LEFT=10
>SETCROPINFO ENABLE 10
<OK
```

GETCROPINFO <ENABLE>, <TOP>, <RIGHT>, <BOTTOM>, <LEFT>, <ALL>

Set the crop info. You can set one or more properties in the same command:

ENABLE set the crop enable [TRUE | FALSE];
TOP get the crop top integer value in pixel;
RIGHT get the crop right integer value in pixel;
BOTTOM get the crop bottom integer value in pixel;
LEFT get the crop left integer value in pixel;
ALL retrieve all properties;

Examples:

```
>LBC_SENDCONTENTCOMMAND "SIGLA", "GETCROPINFO ENABLE TOP RIGHT BOTTOM LEFT"
<ENABLE="TRUE",TOP=10,RIGHT=0,BOTTOM=12,LEFT=11

>LBC_SENDCONTENTCOMMAND "SIGLA", "GETCROPINFO ALL"
<ENABLE="TRUE",TOP=10,RIGHT=0,BOTTOM=12,LEFT=11
```

6.11 SlideShow

CLEARPLAYLIST

Clear the Slideshow files playlist.

Example:

```
>ClearPlaylist
<OK
```

ADDPLAYLISTFILE <FILEPATH>

Add the given file to the playlist at the last position. The file path may refer to either local or remote URL file.

Example:

```
>AddPlayListFile \"C://Datasource//tg.txt\"
<OK

>AddPlayListFile \"http://www.classx.it//prova.txt\"
<OK
```

RESETPLAYLIST

Reset the playlist execution forcing the Slideshow to restart by the first playlist file.

Example:

```
>ResetPlaylist
<OK
```

6.12 NDI Source

SETCROPINFO <ENABLE>, <PARAM0>, <PARAM1>, <PARAM2>, <PARAM3>

Set the crop info. You can set one or more properties in the same command:

ENABLE set the crop enable [TRUE | FALSE];
PARAM0
PARAM1
PARAM2
PARAM3 are the crop integer values in pixel;

This command respects the HTML#margins standard.

If you set 4 values are applied with order top/right/bottom/left
If you set 3 values are applied with order top/left/bottom
If you set 2 values the first value is applied to top/bottom and the second is applied to left/right
If you set 1 values are applied in all sides

Examples:

```
// ENABLE="TRUE",TOP=10,RIGHT=11,BOTTOM=12,LEFT=13
>SETCROPINFO ENABLE 10 11 12 13
<OK

// ENABLE="TRUE",TOP=10,RIGHT=0,BOTTOM=12,LEFT=13
>SETCROPINFO ENABLE 10 11 12
<OK

// ENABLE="TRUE",TOP=15,RIGHT=20,BOTTOM=15,LEFT=20
>SETCROPINFO ENABLE 15 20
<OK

// ENABLE="TRUE",TOP=10,RIGHT=10,BOTTOM=10,LEFT=10
>SETCROPINFO ENABLE 10
<OK
```

GETCROPINFO <ENABLE>, <TOP>, <RIGHT>, <BOTTOM>, <LEFT>, <ALL>

Set the crop info. You can set one or more properties in the same command:

ENABLE set the crop enable [TRUE | FALSE];
TOP get the crop top integer value in pixel;
RIGHT get the crop right integer value in pixel;
BOTTOM get the crop bottom integer value in pixel;
LEFT get the crop left integer value in pixel;
ALL retrieve all properties;

Examples:

```
>LBC_SENDCONTENTCOMMAND "SIGLA", "GETCROPINFO ENABLE TOP RIGHT BOTTOM LEFT"
<ENABLE="TRUE",TOP=10,RIGHT=0,BOTTOM=12,LEFT=11

>LBC_SENDCONTENTCOMMAND "SIGLA", "GETCROPINFO ALL"
<ENABLE="TRUE",TOP=10,RIGHT=0,BOTTOM=12,LEFT=11
```

6.13 SharedFrame

SETCROPINFO <ENABLE>, <PARAM0>, <PARAM1>, <PARAM2>, <PARAM3>

Set the crop info. You can set one or more properties in the same command:

ENABLE set the crop enable [TRUE | FALSE];
PARAM0
PARAM1
PARAM2
PARAM3 are the crop integer values in pixel;

This command respects the HTML#margins standard.

If you set 4 values are applied with order top/right/bottom/left
If you set 3 values are applied with order top/left/bottom
If you set 2 values the first value is applied to top/bottom and the second is applied to left/right
If you set 1 values are applied in all sides

Examples:

```
// ENABLE="TRUE",TOP=10,RIGHT=11,BOTTOM=12,LEFT=13
>SETCROPINFO ENABLE 10 11 12 13
<OK

// ENABLE="TRUE",TOP=10,RIGHT=0,BOTTOM=12,LEFT=13
>SETCROPINFO ENABLE 10 11 12
<OK

// ENABLE="TRUE",TOP=15,RIGHT=20,BOTTOM=15,LEFT=20
>SETCROPINFO ENABLE 15 20
<OK

// ENABLE="TRUE",TOP=10,RIGHT=10,BOTTOM=10,LEFT=10
>SETCROPINFO ENABLE 10
<OK
```

GETCROPINFO <ENABLE>, <TOP>, <RIGHT>, <BOTTOM>, <LEFT>, <ALL>

Set the crop info. You can set one or more properties in the same command:

ENABLE set the crop enable [TRUE | FALSE];
TOP get the crop top integer value in pixel;
RIGHT get the crop right integer value in pixel;
BOTTOM get the crop bottom integer value in pixel;
LEFT get the crop left integer value in pixel;
ALL retrieve all properties;

Examples:

```
>LBC_SENDCONTENTCOMMAND "SIGLA", "GETCROPINFO ENABLE TOP RIGHT BOTTOM LEFT"
<ENABLE="TRUE",TOP=10,RIGHT=0,BOTTOM=12,LEFT=11

>LBC_SENDCONTENTCOMMAND "SIGLA", "GETCROPINFO ALL"
<ENABLE="TRUE",TOP=10,RIGHT=0,BOTTOM=12,LEFT=11
```

6.14 Video Input

SETSOURCEID <SOURCEID>

It changes the ID number of the source on the Video Input

SOURCEID ID number

Example:

```
>LBC_SENDCONTENTCOMMAND "contentName", "SetSourceID 2"
<OK
```

SETVOLUME <SETVOLUME>

Sets the volume to the specified video input source.

SETVOLUME percentage of volume

Example:

```
>LBC_SENDCONTENTCOMMAND "contentName", "SetVolume 10"
<OK
```

NB: this method does not modify in any way the master volume of the source. As soon as the master volume is reset, this volume is reset. (the values of the volume can be viewed on the tooltips of the slider of the VideoInput preview panel). This volume is relatively calculated compared to the master volume.

6.15 Video Input Switcher

SETSOURCEID <SOURCEID>

It changes the current Video Input Switcher ID

SOURCEID ID the number of the current ID. It must be in the range 0-9

Example:

```
>LBC_SENDCONTENTCOMMAND "contentName", "SetSourceID 2"
<OK
```

SETNEXTSOURCEID <SOURCEID>

It changes the next Video Input Switcher ID

SOURCEID ID the number of the next ID. It must be in the range 0-9

Example:

```
>LBC_SENDCONTENTCOMMAND "contentName", "SetNextSourceID 5"
<OK
```

GETTAKEMODE

It returns a string representing the selected Video Input Switcher take mode.

Example:

```
>LBC_SENDCONTENTCOMMAND "contentName", "GetTakeMode"
<TAKEMODE=FADE
```

GETTAKEMODELIST

It returns the list of the available Video Input Switcher take modes.

Example:

```
>LBC_SENDCONTENTCOMMAND "contentName", "GetTakeModeList"
<CUT,FADE,TR1,TR2,TR3,TR4
```

HASFRAMEMARKERS <TAKEMODE>

Check if the given take mode contains frame markers. It returns true if this take mode contains at least 1 frame marker.

TAKEMODE a string representing one of the available take modes

Example:

```
>LBC_SENDCONTENTCOMMAND "contentName", "HasFrameMarkers TR1"
<true
```

TAKE <TAKEMODE>

It takes the a new Video Input source corresponding to the next source ID.

TAKEMODE a string representing one of the available take modes

Example:

```
>LBC_SENDCONTENTCOMMAND "contentName", "Take TR1"
<Ok
```

GETTRANSITIONSTASTUS

It returns the current transition status.

Example:

```
>LBC_SENDCONTENTCOMMAND "contentName", "GetTransitionStatus"
<TRANSITIONSTATUS="TRANSITION_RUNNING"
```

RESUMETRANSITIONFORWARD

It resumes the transition forward if the transition is in the `"TRANSITION_PAUSED"` status.

Example:

```
>LBC_SENDCONTENTCOMMAND "contentName", "ResumeTransitionForward"
<Ok
```

RESUMETRANSITIONBACKWARD

It resumes the transition backward if the transition is in the `"TRANSITION_PAUSED"` status.

Example:

```
>LBC_SENDCONTENTCOMMAND "contentName", "ResumeTransitionBackward"
<Ok
```